

Relatório 3 – Organização de Computadores I – INE5411

Alunas: Ana Luiza Sales Gobbi (24105453), Fernanda Bertotti Guedes (24100601) e

Laísa Ágathe Bridi Dacroce (24100598)

Data: 30/04/25

1 INTRODUÇÃO

O presente relatório visa discorrer sobre os exercícios propostos no Laboratório 3 da disciplina de Organização de Computadores I. Nessas atividades, foi possível exercitar conceitos relacionados a ponto flutuante, em especial, instruções e registradores de ponto flutuante de precisão dupla. Nas seções abaixo, serão descritas as soluções desenvolvidas, assim como as principais dificuldades experienciadas.

2 EXERCÍCIOS PROPOSTOS

2.1 EXERCÍCIO 1

O primeiro exercício teve como objetivo implementar, em assembly MIPS, um procedimento capaz de calcular a raiz quadrada de um número real utilizando o método iterativo de Newton. Para isso, o número 'x' e o número de iterações 'n' foram lidos do teclado, sendo 'x' um valor de ponto flutuante em precisão dupla e 'n' um valor inteiro.

A estimativa inicial foi definida como 1.0, conforme o exercício pediu. O procedimento chamado `raiz_quadrada` foi responsável por executar o loop iterativo 'n' vezes, armazenando o valor atualizado da estimativa a cada ciclo realizado. Todos os cálculos foram realizados utilizando registradores e instruções de ponto flutuante em precisão dupla, como `div.d`, `add.d` e `sqrt.d`, conforme exigido. Ao final do processo, o valor da estimativa foi comparado com a raiz quadrada calculada pela instrução `sqrt.d`, e o erro absoluto entre os dois foi armazenado no registrador `$f4`.

Além disso, foram realizados testes com diferentes valores de 'x' e 'n', nos quais foi possível observar que quanto maior o valor de 'n', mais precisa é a estimativa retornada. Também foi constatado que, quanto maior o valor de 'x', maior deve ser o número de iterações 'n' para que se atinja uma boa precisão no resultado.

Por exemplo, considerando dois casos: $x' = 25$ e $x'' = 100$. As imagens abaixo, retiradas da aba do Coprocessador 1 do MARS, apontam que, para x' , um valor de $n = 7$ já é suficiente para que o erro absoluto entre o procedimento `raiz_quadrada` e `sqrt.d` seja igual a 0.0.

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	3.122258E-27	5.000023178253949
\$f1	2.3125014	
\$f2	0.0	5.0
\$f3	2.3125	
\$f4	1.7446117E36	2.317825394904105E-5
\$f5	0.48496905	
\$f6	0.0	2.0
\$f7	2.0	
\$f8	0.0	0.0
\$f9	0.0	
\$f10	0.0	0.0
\$f11	0.0	
\$f12	0.0	25.0
\$f13	2.890625	
\$f14	0.0	0.0
\$f15	0.0	
\$f16	0.0	0.0
\$f17	0.0	
\$f18	0.0	0.0
\$f19	0.0	
\$f20	0.0	0.0
\$f21	0.0	
\$f22	0.0	0.0
\$f23	0.0	
\$f24	0.0	0.0
\$f25	0.0	
\$f26	0.0	0.0
\$f27	0.0	
\$f28	0.0	0.0
\$f29	0.0	
\$f30	0.0	0.0
\$f31	0.0	

Figura 1 – $N = 5$ é insuficiente para que erro absoluto de $X = 25$ seja 0.0.

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	0.0	5.0
\$f1	2.3125	
\$f2	0.0	5.0
\$f3	2.3125	
\$f4	0.0	0.0
\$f5	0.0	
\$f6	0.0	2.0
\$f7	2.0	
\$f8	0.0	0.0
\$f9	0.0	
\$f10	0.0	0.0
\$f11	0.0	
\$f12	0.0	25.0
\$f13	2.890625	
\$f14	0.0	0.0
\$f15	0.0	
\$f16	0.0	0.0
\$f17	0.0	
\$f18	0.0	0.0
\$f19	0.0	
\$f20	0.0	0.0
\$f21	0.0	
\$f22	0.0	0.0
\$f23	0.0	
\$f24	0.0	0.0
\$f25	0.0	
\$f26	0.0	0.0
\$f27	0.0	
\$f28	0.0	0.0
\$f29	0.0	
\$f30	0.0	0.0
\$f31	0.0	

Figura 2 – $N = 7$ é o primeiro valor que faz com que o erro absoluto de $X = 25$ seja 0.0.

Já para $x'' = 100$, é necessário um valor de n igual a 8 para que o erro absoluto seja igual a 0.0, pois ainda há uma pequena variação entre o resultado do procedimento `raiz_quadrada` e `sqrt.d` quando $n = 7$, conforme as imagens abaixo.

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	1.10359E-40	10.000000000139897
\$f1	2.5625	
\$f2	0.0	10.0
\$f3	2.5625	
\$f4	0.0	1.3989698288696673E-10
\$f5	0.11095083	
\$f6	0.0	2.0
\$f7	2.0	
\$f8	0.0	0.0
\$f9	0.0	
\$f10	0.0	0.0
\$f11	0.0	
\$f12	0.0	100.0
\$f13	3.390625	
\$f14	0.0	0.0
\$f15	0.0	
\$f16	0.0	0.0
\$f17	0.0	
\$f18	0.0	0.0
\$f19	0.0	
\$f20	0.0	0.0
\$f21	0.0	
\$f22	0.0	0.0
\$f23	0.0	
\$f24	0.0	0.0
\$f25	0.0	
\$f26	0.0	0.0
\$f27	0.0	
\$f28	0.0	0.0
\$f29	0.0	
\$f30	0.0	0.0
\$f31	0.0	

Figura 3 – $N = 7$ é insuficiente para zerar erro absoluto com $X = 100$.

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	0.0	10.0
\$f1	2.5625	
\$f2	0.0	10.0
\$f3	2.5625	
\$f4	0.0	0.0
\$f5	0.0	
\$f6	0.0	2.0
\$f7	2.0	
\$f8	0.0	0.0
\$f9	0.0	
\$f10	0.0	0.0
\$f11	0.0	
\$f12	0.0	100.0
\$f13	3.390625	
\$f14	0.0	0.0
\$f15	0.0	
\$f16	0.0	0.0
\$f17	0.0	
\$f18	0.0	0.0
\$f19	0.0	
\$f20	0.0	0.0
\$f21	0.0	
\$f22	0.0	0.0
\$f23	0.0	
\$f24	0.0	0.0
\$f25	0.0	
\$f26	0.0	0.0
\$f27	0.0	
\$f28	0.0	0.0
\$f29	0.0	
\$f30	0.0	0.0
\$f31	0.0	

Figura 4 – $N = 8$ é suficiente para que erro absoluto seja igual a 0.0 com $X = 100$.

As principais dificuldades enfrentadas nesse exercício foram relacionadas ao uso correto das instruções de ponto flutuante em dupla precisão e à manipulação dos registradores adequados para operações com valores double. Além disso, foi necessário compreender com clareza como organizar as chamadas de syscall para entrada de dados e retorno de procedimentos.

2.2 EXERCÍCIO 2

Para limitar o cálculo aos primeiros 20 termos da série, foi inicializada uma palavra denominada N no segmento de dados, com o valor 20. Também foram inicializadas as variáveis X e seno, que representam, respectivamente, o argumento do procedimento do código e o resultado final esperado.

Ao analisar a equação, percebemos um padrão na variação dos expoentes de X e dos denominadores de cada termo: ambos eram acrescidos em dois a cada iteração. Para simplificar o código, optamos, então, por armazenar o valor inicial de $2n+1$ em um registrador \$f6 e incrementá-lo em duas unidades a cada incremento da nossa variável de controle i, a ser comparada com o valor de N.

O procedimento calcula_seno, então, recebe como argumento o valor de X e implementa um loop com o cálculo do denominador (fatorial) e do numerador (potenciação) do elemento do somatório, que é multiplicado por 1 ou -1, dependendo da paridade da variável de controle na iteração.

Não foi possível comprovar o correto funcionamento do código, pois projetamos a lógica dos loops de fatorial e potenciação com base em valores de ponto flutuante de precisão dupla, convertidos para inteiros apenas no âmbito da instrução de desvio condicional utilizada. Isso fez com que o código executasse indefinidamente em loop, por conta da imprecisão dos valores de comparação com as condições estabelecidas nos branches. Por meio de pesquisas, foi possível perceber que o ideal seria realizar toda a operação de fatorial e potenciação com registradores de inteiros, e realizar a conversão do resultado obtido apenas no final, após o loop. Contudo, por conta de limitações de tempo, não foi possível implementar essa solução funcional. A imagem abaixo retrata o estado da aba Coprocessador 1 do MARS, após uma tentativa falha de verificar o procedimento para $X = 90$. É possível perceber que o código parou na execução do fatorial, conforme mencionado.

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	0.0	90.0
\$f1	3.3515625	
\$f2	0.0	0.0
\$f3	0.0	
\$f4	0.0	1.0
\$f5	1.875	
\$f6	0.0	1.0
\$f7	1.875	
\$f8	0.0	-0.0
\$f9	-0.0	
\$f10	0.0	-667054.0
\$f11	-10.272305	
\$f12	0.0	90.0
\$f13	3.3515625	
\$f14	NaN	2.1216662225E-314
\$f15	0.0	
\$f16	0.0	1.0
\$f17	1.875	
\$f18	0.0	0.0
\$f19	0.0	
\$f20	0.0	0.0
\$f21	0.0	
\$f22	0.0	0.0
\$f23	0.0	
\$f24	0.0	0.0
\$f25	0.0	
\$f26	0.0	0.0
\$f27	0.0	
\$f28	0.0	0.0
\$f29	0.0	
\$f30	0.0	0.0
\$f31	0.0	

Figura 5 – Falha na execução do código do exercício 2

Dentre os desafios encontrados nesse exercício, certamente se destaca o uso de instruções e registradores de ponto flutuante de precisão dupla, tendo em vista que foi preciso refatorar o código seguidamente para adaptá-lo às restrições que isso impõe. Especialmente, a impossibilidade de realizar operações como `andi` e `li` se mostrou desafiadora. Para contorná-la, utilizamos instruções como `mtc1`, `cvt.d.w`, `mfc1` e `cvt.w.d`. Com esse exercício, foi possível perceber o verdadeiro adicional de complexidade trazido pelo ponto flutuante de precisão dupla, pois em diversos momentos ficamos em dúvida se seria necessário armazenar um valor em um registrador do tipo `$t` ou do tipo `$f`.